Completion-Aware Bus Holding Using Hybrid Offline and Simulation-Based Reinforcement Learning

Yifan Zhang, Liang Zheng, and Qifan Zhang

***Abstract*—Learning a bus holding policy in a fast simulator avoids exploratory control of an operating fleet, but reducing passenger time alone can leave more journeys unfinished. We develop a completion-aware controller that combines logged microscopic simulation data with further learning in a calibrated event-based simulator. A shared policy chooses additional holding times for 12 directional services. It represents zero hold explicitly and learns from an exact accounting of waiting and in-vehicle time. A simple execution rule keeps a lightly loaded bus at a stop for 60 seconds when passenger arrivals continue and no following bus is active, within a specified part of the service horizon. We test this controller against an otherwise identical controller with a shorter rule active over a narrower time window. Evaluation covers 1,350 fresh microscopic simulation episodes; the primary comparison uses ten independent environment blocks, three demand levels, and five trained policies. The extended rule reduces generalized passenger time from 2,484.86 to 2,392.73 seconds per passenger who starts a journey, a 3.71% reduction. Average waiting time falls by 56.23 seconds, while in-vehicle time rises by 20.34 seconds. Journey completion increases from 96.185% to 96.650%. Both prespecified criteria are met in all ten blocks. These results support the rule's added benefit for the tested learned controller and make its passenger tradeoff explicit.**

***Index Terms*—Optimization and control, data-based approaches, public transportation, road transportation, bus holding, hybrid reinforcement learning.**

I. Introduction

A BUS delayed on a busy route encounters more waiting passengers and spends longer serving them. The following bus then encounters fewer passengers and catches up. This feedback can turn a small delay into a long gap followed by several buses arriving together [1], [2]. Holding a bus at a stop can restore spacing and reduce waiting downstream, but it also delays people already on board. A useful controller must balance these costs and preserve passengers' opportunities to complete their journeys [3].

Reinforcement learning (RL) offers a way to make these decisions from traffic, headway, and passenger information. Existing studies have considered dynamic holding, asynchronous decisions, and interactions between services sharing a corridor [4]–[7]. Their practical use raises a familiar question: where should a policy acquire its experience? Exploratory actions on an operating fleet affect passengers directly. Logged data avoid this problem but provide limited coverage of alternative actions. A fast simulator supplies additional experience, although its travel times, boarding processes, and service interactions may differ from those in the target environment.

Hybrid Offline-and-Online (H2O) RL addresses this setting by combining a fixed target-domain dataset with interaction in a different simulator [8]. H2O+ extends the approach to different offline and online learners [9]. We adapt this framework to bus holding using microscopic simulation records for offline learning and a calibrated event-based model for further training. All online interaction during training occurs in the latter model. The evaluation target is also a simulator; no operating fleet is used for exploration or testing.

A second problem concerns the objective itself. Over a fixed service horizon, a controller can lower accrued passenger time while leaving more journeys unfinished. Keeping a lightly loaded bus moving may benefit its current passengers yet remove a boarding opportunity for people arriving shortly afterward. Their waiting time is counted until the horizon, but the rest of an unfinished journey lies outside that accounting period. During development, improvements in the time objective sometimes coincided with unacceptable reductions in completion.

We therefore combine the learned policy with an explicit passenger-service rule. After ordinary boarding and alighting, a lightly loaded bus waits for 60 s if a preceding bus has served the stop, no following bus is active, and the current eligible arrival rate is positive. The rule applies only during a fixed part of the service horizon. Its purpose is to retain a boarding opportunity where the learned time objective alone may provide insufficient protection.

The study develops a shared, event-based holding controller with an explicit zero-hold action and passenger-time accounting across simultaneous requests. It combines hybrid offline and simulator-based learning with a transparent rule for retaining boarding opportunities. A matched evaluation then tests whether extending this rule reduces passenger time while preserving journey completion.

The central comparison holds the trained policy parameters and prior action calibration fixed. It changes an earlier 20-s rule, active during a narrower window, to the 60-s rule active over a wider window. Across ten independent test blocks, generalized passenger time falls by 92.12 s per departed passenger and completion rises by 0.466 percentage points on average. A decomposition shows the practical tradeoff: less

Yifan Zhang and Liang Zheng are with Central South University, Changsha, Hunan, China (e-mail: erzhu419@gmail.com; 204201048@csu.edu.cn; zhengliang@csu.edu.cn).

Qifan Zhang is with the Faculty of Information Technology, Monash University Malaysia, Malaysia (e-mail: zhangqifan0419@gmail.com).

waiting outweighs additional time on board.

II. Related Work

A. Bus Holding and Passenger Service

Classical bus control uses headway or schedule information to limit the propagation of disturbances. Daganzo [2] analyzes headway-based holding, whereas Xuan *et al.* [10] study schedule reliability through dynamic linear control. Passenger-oriented optimization explicitly weighs waiting against additional time on board [3]. Real-time methods incorporate changing running times and demand [11], and predictive control uses forecasts of headway evolution to choose holding actions [12].

RL has extended this line of work to more complex operational states. Alesiani and Gkiotsalitis [4] study high-frequency holding, and Wang and Sun [5] develop a multi-agent controller. Later work handles asynchronous decisions [6], multiline services sharing a corridor [7], and robustness through distributional multi-agent learning [13]. These studies already establish asynchronous and coordinated bus control as active research topics. Our contribution concerns how a hybrid-learned policy is represented, executed, and evaluated when both passenger time and finite-horizon completion matter.

B. Learning from Offline Data and Further Interaction

Soft actor-critic (SAC) learns an entropy-regularized policy from replayed interactions [14], [15]. Offline methods address the difficulty of evaluating actions poorly represented in a fixed dataset. Conservative Q-learning lowers estimates for unsupported actions [16], while implicit Q-learning (IQL) learns an expectile value and extracts a policy by advantage-weighted regression [17]. We use IQL for initialization and SAC-style objectives during simulator training.

Continuing an offline policy can be unstable because the replay distribution and value estimates change. Balanced replay and critic ensembles [18], value calibration [19], and policy expansion [20] provide different responses. Reinforcement learning with prior data mixes existing and newly collected experience [21]; warm-start RL instead uses a short initialization period before continuing without offline replay [22]. We retain offline replay and gradually release actor updates. These choices adapt existing learning ideas to the present control setting.

Offline training followed by online refinement has also been explored for coordinating ride-pooling and public transit [23]. Consequently, the present study does not claim to introduce offline-to-online learning to transportation. Its focus is fixed-route holding with asynchronous stop events, a separate training simulator, and an explicit completion rule.

C. Learning across Simulator Dynamics

Off-dynamics RL estimates a conditional dynamics ratio using domain classifiers and uses it to correct rewards [24]. H2O and H2O+ combine target-domain records with simulation while accounting for differences in dynamics [8], [9]. Such correction is useful only if the estimated quantities generalize beyond the training trajectories.

We retain H2O+'s mixed-data value-learning structure but assign equal weights to simulator transitions. The domain classifiers did not generalize adequately to held-out trajectories and are used only to diagnose the dynamics mismatch. Section IV describes this choice and its empirical basis.

III. Bus Holding Problem

A. Decisions, Observations, and Actions

One shared policy controls 12 directional services over a horizon of $T = 24{,}000$ s. A decision is requested at a nonterminal stop after the bus completes its mandatory boarding and alighting service. The action specifies additional holding only. During that hold, later arrivals may board if capacity remains.

Several buses may request control at the same physical time. We order these requests by service and vehicle identifier, choose their actions sequentially, and advance the simulator only after the full group has been processed. At each step, the policy sees the pending requests and actions already selected for the group. This event-based representation has variable transition durations, as in a semi-Markov decision process [25].

The observation contains five group-level variables and up to 16 padded event records, or slots. The group-level variables are the current time, number of requests, position of the current request, and total waiting and onboard passenger counts. A slot describes the service and stop, forward and backward headways and their scheduled targets, eligible waiting passengers, onboard load, eligible passenger-arrival rate, base service time, and indicators of the preceding and following buses. Passenger quantities are normalized using capacity. Each normalized slot contains an activity flag, 12 local features, and the previously selected action, giving $5 + 16(14) = 229$ inputs. Inactive slots are masked. We denote this encoded observation at decision k by s_k, a generic such state by s, and its successor by s'. This compact observation omits parts of the traffic and passenger histories; its Markov sufficiency is an assumption.

Let $a_k \in [-1, 1]$ be the normalized action at decision k. The additional hold is

$$h_k = \frac{a_k + 1}{2} h_{\max}, \qquad h_{\max} = 60 \text{ s}. \tag{1}$$

Thus $a_k = -1$ is exact zero hold; a denotes a generic normalized action below. The cap concerns additional holding and does not replace mandatory passenger service.

B. Passenger-Time Objective

Let t_k be the physical time of decision k, and let Δt_k be the elapsed time until the next decision. Within a simultaneous group, intermediate transitions have $\Delta t_k = 0$. Only its last transition carries the interval to the next physical event or the horizon.

Denote the numbers of waiting and onboard passengers at time u by $N_{\mathrm{w}}(u)$ and $N_{\mathrm{v}}(u)$. The physical reward is the negative accrued generalized passenger time,

$$r_k = -\int_{t_k}^{t_k + \Delta t_k} [2N_{\mathrm{w}}(u) + N_{\mathrm{v}}(u)] \, \mathrm{d}u. \tag{2}$$

Waiting time receives weight two and in-vehicle time weight one. Zero-duration transitions have zero reward. Since positive-duration intervals do not overlap, summing their costs counts each waiting or in-vehicle passenger-second once. The initial interval before any controllable event is policy independent and is added when evaluating the full horizon.

The return formed from (2) is undiscounted. Let $z_k \in \{0,1\}$ equal one when transition k ends the episode. The Bellman continuation is $1 - z_k$; no extra discount is applied for the duration between events. The learning objective is to maximize expected return in the microscopic target environment, using a fixed offline dataset and a limited number of interactions in the faster training simulator.

For evaluation, let N_{dep} count all passengers who start their journeys and N_{cmp} count those who finish by T. We report

$$W = \frac{1}{N_{\mathrm{dep}}} \int_0^T N_{\mathrm{w}}(u)\,\mathrm{d}u, \qquad V = \frac{1}{N_{\mathrm{dep}}} \int_0^T N_{\mathrm{v}}(u)\,\mathrm{d}u, \tag{3}$$

$$Y = 2W + V, \qquad R_{\mathrm{cmp}} = \frac{N_{\mathrm{cmp}}}{N_{\mathrm{dep}}}, \qquad U = N_{\mathrm{dep}} - N_{\mathrm{cmp}}. \tag{4}$$

In (3), W and V are accrued waiting and in-vehicle seconds per departed passenger. In (4), Y is generalized seconds per departed passenger, R_{cmp} is the journey-completion rate, and U is the unfinished-passenger count. The denominator includes passengers whose journeys remain unfinished. Their accrued waiting and in-vehicle time also remain in the numerator; time beyond T is not observed.

Within a fixed demand realization, minimizing Y and maximizing the physical return give the same policy ordering. Across different demand levels, the total-cost training objective and the equally averaged per-passenger evaluation need not give the same weights to each scenario. Completion is measured separately because a low accrued-time objective does not guarantee that journeys finish.

C. Offline Records and Training Simulator

The offline dataset contains tuples $(s_k, a_k, r_k, s_{k+1}, \Delta t_k, z_k)$ from the microscopic Simulation of Urban MObility (SUMO) environment [26]. The learner adds experience only from a calibrated event-based simulator. Both environments share service routes, stop sequences, timetables, capacities, passenger journeys, and the observation, action, and reward definitions above.

The event-based model uses calibrated segment running times and boarding processes in place of microscopic road traffic. Passenger transfers are released after the preceding leg actually finishes, so a later leg cannot be boarded before the passenger reaches it. Matching these operational definitions allows replay to be combined, but does not make the two environments' transition dynamics identical. Their response agreement is checked before policy evaluation, as described in Section V.

IV. Learning and Execution

Fig. 1 separates learning from execution. The learner produces a holding proposal. Two deterministic operations then calibrate its duration and apply the completion rule. All parameters are fixed during target evaluation.

A. Representing Exact Zero Hold

The actor, critic, and value networks use separate structured encoders. A shared network maps each event slot to 128 features. The current slot, active-slot mean and maximum, earlier-slot mean, and later-slot mean are combined with the group-level variables to obtain a 256-feature representation. The heads then use two hidden layers of 256 units. There are two critic heads and their target copies.

Exact zero hold is common and operationally distinct from a small positive hold. We therefore use a hurdle policy: a binary decision determines whether to hold, and a continuous distribution determines the positive duration. Let π_ϕ denote this policy, with actor parameters ϕ, holding probability $p_\phi(s)$, latent normal mean $\mu_\phi(s)$, and positive standard deviation $\sigma_\phi(s)$. With $\mathrm{sigmoid}(x) = 1/(1+e^{-x})$, its normalized action is generated by

$$\begin{aligned} &a = -1 && \text{with probability } 1 - p_\phi(s), \\ &a = 2\,\mathrm{sigmoid}(\zeta) - 1 && \text{otherwise}, \\ &\zeta \sim \mathcal{N}(\mu_\phi(s), \sigma_\phi^2(s)). \end{aligned} \tag{5}$$

Here ζ is the latent normal variable. The positive branch in (5) has density

$$f_\phi(a \mid s) = \frac{2\,\mathcal{N}\Big(\log \frac{1+a}{1-a}; \mu_\phi(s), \sigma_\phi^2(s)\Big)}{1 - a^2}, \quad -1 < a < 1. \tag{6}$$

In (6), $\mathcal{N}(x; \mu, \sigma^2)$ denotes the normal density evaluated at x, and the Jacobian follows directly from the logistic transformation. Likelihoods use the point mass at -1 together with the continuous density on $(-1, 1)$. We write the density with respect to this common point-mass-plus-Lebesgue measure as $q_\phi(a \mid s)$: it equals $1 - p_\phi(s)$ at zero hold and $p_\phi(s) f_\phi(a \mid s)$ on the positive branch. Positive actions are clipped near the numerical endpoints when evaluating likelihoods, including logged maximum holds; stored actions and physical holds are unchanged by this likelihood calculation.

Evaluation is deterministic: if $p_\phi(s) < 0.5$, the proposal is $h_\phi(s) = 0$; otherwise it is $h_\phi(s) = h_{\max}\,\mathrm{sigmoid}(\mu_\phi(s))$, using the action-to-hold map in (1). The latter is the positive branch's median hold, which generally differs from its mean.

B. Offline Initialization and Simulator Learning

Five actors are initialized with 50,000 IQL updates [17]. Let $\mathcal{D}_{\mathrm{off}}$ and $\mathcal{D}_{\mathrm{sim}}$ denote the empirical sampling distributions of offline and simulator replay. We write $\mathbb{E}$ for expectation and $\mathbb{I}$ for the indicator of a stated condition. Let $\overline{Q}_{\min}(s, a)$ be the minimum of the two target critics and $V_\nu(s)$ the expectile value with parameters ν. Rewards are multiplied by $\kappa = 10^{-4}$ inside the trainer for numerical conditioning; the stored data and reported outcomes retain physical units. All critics and values below use these scaled rewards. For $A(s, a) = \overline{Q}_{\min}(s, a) - V_\nu(s)$, the value minimizes the expectile loss

$$L_V = \mathbb{E}_{\mathcal{D}_{\mathrm{off}}}[\,|0.7 - \mathbb{I}\{A < 0\}|A^2\,]. \tag{7}$$

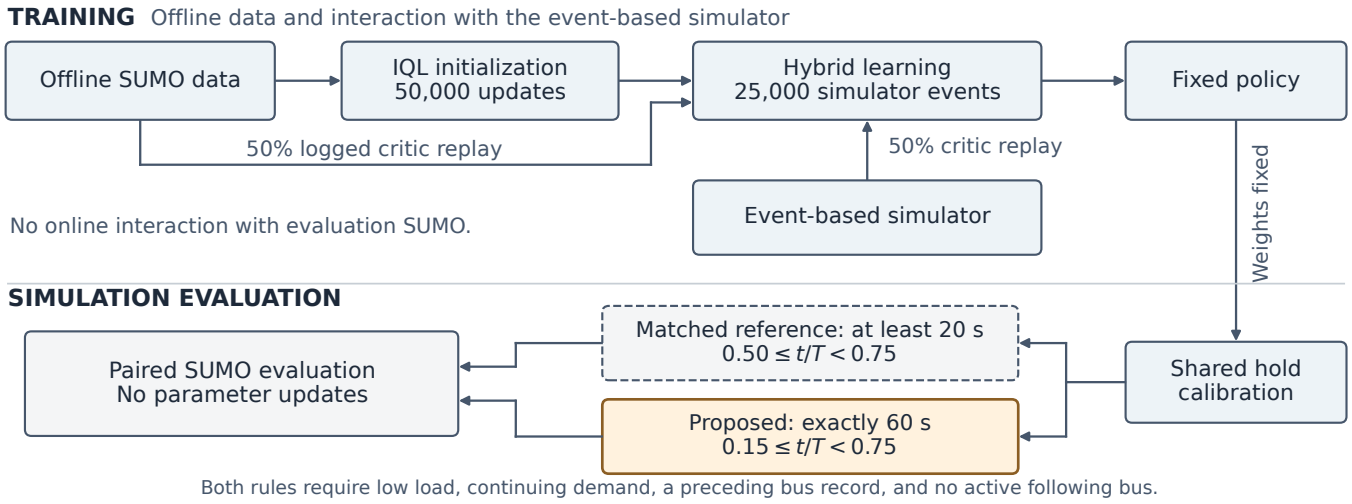


Fig. 1. Learning and evaluation of the bus holding controller. Offline microscopic records and further interaction in an event-based simulator produce a fixed actor. The primary comparison uses that same actor and hold calibration in both controllers, changing the completion rule's duration and activation window. Target evaluation supplies no training updates.

With $A = A(s, a)$, the actor minimizes the negative logged-action likelihood with weight

$$w(A) = \min\{\exp(3A), 100\}, \qquad L_{\pi,\text{off}} = -\mathbb{E}_{\mathcal{D}_{\text{off}}}[w(A) \log q_\phi(a \mid s)]. \tag{8}$$

Here L_V in (7) is the value-fitting loss, $w(A)$ is the capped regression weight, and $L_{\pi,\text{off}}$ in (8) is the offline actor loss.

The initialized policy first collects 5,000 simulator events without learning. Subsequent critic minibatches combine offline and simulator transitions equally. Following the H2O+ mixed-backup construction [9], define the soft bootstrap B_{soft} and critic target y_k, with entropy temperature $\alpha > 0$, by

$$B_{\text{soft}}(s') = \mathbb{E}_{a' \sim \pi_\phi(\cdot|s')}[\overline{Q}_{\min}(s', a') - \alpha \log q_\phi(a' \mid s')], \tag{9}$$

$$y_k = \kappa r_k + (1 - z_k)[0.1 V_\nu(s_{k+1}) + 0.9 B_{\text{soft}}(s_{k+1})]. \tag{10}$$

Here a' is a successor-state action in (9); y_k in (10) combines the physical reward with the mixed bootstrap. The critic uses a Huber loss with threshold one. The expectile value continues to be updated from the offline transitions during simulator training. The actor minimizes

$$L_{\pi,\text{sim}} = \mathbb{E}_{s \sim \mathcal{D}_{\text{sim}},\, a \sim \pi_\phi(\cdot|s)}[\alpha \log q_\phi(a \mid s) - Q_{\min}(s, a)], \tag{11}$$

where $L_{\pi,\text{sim}}$ denotes the simulator-phase actor loss and $Q_{\min}$ is the minimum of the current critics, distinct from the target-critic minimum $\overline{Q}_{\min}$. The entropy temperature α is learned with target entropy zero [14], [15]. The binary branch is summed exactly; the positive branch uses one reparameterized sample per state.

Actor updates using (11) and temperature updates remain frozen until 10,000 simulator events. The actor learning-rate multiplier increases from approximately 0.1 to 1.0 over events 10,000–15,000, then decreases linearly to 0.25 over events 15,000–25,000. After each actor update, parameter interpolation with coefficient 5×10^{-5} pulls the actor toward its own 10,000-event parameters. This limits drift while the critics adapt. Temperature updates resume at their nominal rate.

TABLE I
TRAINING SETTINGS

Setting	Value
Slot / fused representation	128 / 256
Hidden layers in each head	256, 256
Critic / actor batch size	256 / 128
Offline fraction of critic batch	0.5
Adam learning rate	3×10^{-4}
Target-network update rate	0.005
Log standard deviation bounds	$[-5, 2]$
Initial entropy temperature	1.0
Reward multiplier	10^{-4}
IQL expectile / weight cap	0.7 / 100
Offline / simulator-training updates	50,000 / 20,000
Nominal simulator-event budget	25,000
Independent training seeds	5

Table I gives the remaining settings. Each policy receives 50,000 updates from offline data and 20,000 updates during simulator training. The nominal simulator budget is 25,000 events; two of the five runs end at 24,999 to avoid splitting a simultaneous decision group. We use only the final scheduled checkpoint, without selecting among checkpoints on target outcomes.

C. Handling Unreliable Dynamics Weights

Joint classifiers observe $(s, a, s', \Delta t/T)$, and marginal classifiers observe (s, a). Their logit difference is intended to estimate the log conditional dynamics ratio between logged microscopic and training-simulator transitions [9], [24]. Validation holds out complete trajectories rather than randomly splitting correlated rows.

The mean joint classification area under the receiver-operating characteristic curve is 0.9477 on training data but 0.4520 on held-out data. The mean raw importance-weight effective sample size is only 6.15% of the validation sample. The final learner therefore assigns unit weights to simulator critic losses. The classifiers remain diagnostic and do not

alter the updates in (10). We refer to this learner as H2O+-based to identify its mixed-data construction while making this implementation choice explicit.

D. Calibrating the Holding Proposal

At physical time t, let $\rho = t/T$ denote the horizon fraction. Denote forward and backward headways by H_f, H_b, and their scheduled targets by H_f^*, H_b^*, all in seconds. The indicator $I_f = 1$ means that the preceding scheduled bus has completed base service at this stop. The indicator $I_b = 1$ means that the following scheduled bus is currently active. An inactive follower can still be scheduled to depart later.

The headway-based duration is

$$h_H(s) = \mathrm{clip}\left(\frac{I_b(H_b - H_b^*) - I_f(H_f - H_f^*)}{2}, 0, 60\right). \tag{12}$$

The calibrated proposal is

$$h_{\mathrm{base}}(s) = \begin{cases} \min\{h_\phi(s), \max[h_\phi(s)/8, h_H(s)]\}, & \rho < 0.75, \\ h_\phi(s)/8, & \rho \geq 0.75. \end{cases} \tag{13}$$

In (12), $\mathrm{clip}(\cdot, 0, 60)$ truncates its argument to $[0, 60]$ s. The calibrated proposal in (13) reduces excessive learned holding while retaining a headway response during the first three quarters of the horizon. These fixed choices were made during development and are shared by the two controllers in the main comparison.

E. Completion Rule and Matched Reference

Let n_{on} be the onboard passenger count, $c > 0$ the bus capacity, and λ the current eligible passenger-arrival rate in passengers per second. The rate is obtained from hourly origin–destination demand and scheduled service frequencies, rather than a learned forecast. For horizon-fraction bounds $0 \leq \rho_{\mathrm{lo}} < \rho_{\mathrm{hi}} \leq 1$, define

$$G_{[\rho_{\mathrm{lo}}, \rho_{\mathrm{hi}})}(s) = \mathbb{I}\{\rho_{\mathrm{lo}} \leq \rho < \rho_{\mathrm{hi}},\ I_f = 1,\ I_b = 0\} \times \mathbb{I}\{\lambda > 0,\ c > 0,\ n_{\mathrm{on}}/c \leq 0.25\}. \tag{14}$$

The binary guard in (14) selects the states where the completion rule applies. The completion-aware controller executes

$$h_{\mathrm{final}}(s) = \begin{cases} 60, & G_{[0.15, 0.75)}(s) = 1, \\ h_{\mathrm{base}}(s), & \text{otherwise.} \end{cases} \tag{15}$$

Thus the rule applies between 3,600 and 18,000 s. It leaves a lightly loaded bus available for arriving passengers when no following vehicle is active. The 60-s bound limits the added action; it does not guarantee that every journey will finish.

The matched reference uses the identical actor and calibration but retains the earlier service rule,

$$h_{\mathrm{ref}}(s) = \begin{cases} \max\{h_{\mathrm{base}}(s), 20\}, & G_{[0.50, 0.75)}(s) = 1, \\ h_{\mathrm{base}}(s), & \text{otherwise.} \end{cases} \tag{16}$$

This reference already includes a service rule, so the comparison measures the added effect beyond that rule. It changes both the activation window and the holding floor, testing the complete rule change rather than identifying the separate effect of either parameter.

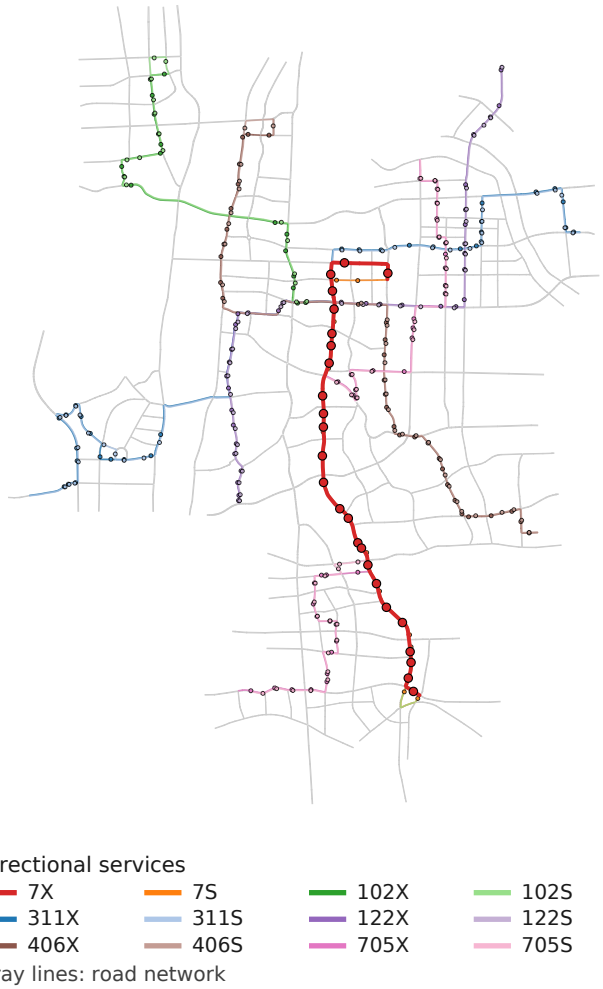


Fig. 2. Road network and 12 directional services in the benchmark. The thicker route is highlighted for visibility; the controller acts on all services. Microscopic and event-based environments use the same route and stop definitions.

Controller design used a development set, with some scenarios reused across iterations. Before final evaluation, we fixed both controllers, all five trained policies, the test scenarios, the outcome measures, and the statistical analysis. The final test set was used only for evaluation.

V. Experimental Design

A. Network, Data, and Independent Test Blocks

The benchmark is a simulated urban bus network with 12 directional services for routes 7, 102, 122, 311, 406, and 705 (Fig. 2). Every 24,000-s episode has 389 scheduled vehicle trips. Demand multipliers are 0.75, 1.00, and 1.25. Passenger journey-start times receive independent uniform perturbations of at most 120 s, clipped to each source route's original departure-time support. The road network, bus timetable, capacities, and social traffic remain fixed.

Offline collection crosses ten environment blocks, three demand levels, and eight behavior policies, yielding 240

complete episodes and 3,284,718 transitions. The behaviors comprise zero hold, two random hurdle policies, four fixed-gain schedule-adjusted headway policies, and a myopic passenger-time policy. They provide varied actions and are not assumed to be experts. The replay contains 240 terminal records and 1,025,206 zero-duration transitions. Across all 240 episodes, the recorded rewards and passenger-time integrals give the same total cost.

For final evaluation, ten new environment blocks are crossed with three demand levels and five actor seeds. A block uses one demand-realization seed and one microscopic-traffic seed shared across its demand levels. These ten blocks are disjoint from training and development. Within each block and demand level, all controllers encounter the same external conditions.

Nine controllers are evaluated, producing $10 \times 3 \times 5 \times 9 = 1{,}350$ episodes. The primary comparison comprises the 300 episodes from (15) and (16), or 150 matched pairs. The remaining seven controllers provide descriptive context. The independent unit for inference is the environment block, not a transition, an actor seed, or an individual episode.

B. Agreement between Training and Target Simulators

Before final evaluation, the event-based model was compared with microscopic simulation in 30 matched scenarios. Under zero holding, the largest point differences were 4.61% for generalized passenger time, 8.75% for waiting time, 1.66% for in-vehicle time, 0.43 percentage points for completion, and 1.04% for completed vehicle-trip duration. Prespecified tolerances were 15% for generalized and waiting time, 10% for in-vehicle time, 3 percentage points for completion, and 5% for trip duration. Passenger demand and decision count had separate 3% and 5% tolerances.

A second check applied seven nonzero holding interventions to those scenarios and compared their changes from zero hold. The largest absolute endpoints of the 95% intervals were 0.574 percentage points for completion and 3.90%, 4.98%, and 2.00% for normalized generalized, waiting, and in-vehicle time responses. Time responses are normalized by the corresponding SUMO zero-hold means. Their respective tolerances were 3 percentage points and 10%. All prespecified response checks passed. The intervals used 10,000 bootstrap resamples of episode quantities within demand levels, sampling the two simulators separately. These checks establish agreement for the tested operating conditions and interventions, rather than for all states a learned policy may visit.

Offline records were collected with SUMO 1.27.1 and Python 3.10.12. Evaluation used SUMO 1.25.0 through libsumo, with Python 3.8.20. Every controller uses the same evaluation runtime. Software-version effects were not evaluated separately.

C. Comparison Controllers

The primary reference uses the same five trained policies and hold calibration as the proposed controller. Additional comparisons include a transit-adapted H2O+ baseline, online-only SAC, the learned policy without execution rules, and four intermediate training variants. These variants alter the data used for actor updates, the timing of those updates, the learning-rate schedule, and parameter anchoring. Their results are reported descriptively alongside the primary comparison.

The transit-adapted H2O+ control uses active dynamics weights; the final learner and subsequent stabilization variants use unit weights. Online-only SAC has the same nominal simulator-event budget but no 50,000-update offline initialization. Therefore, its training computation is not matched to that of the hybrid learner.

D. Prespecified Statistical Analysis

For each environment block b, we first average the 15 paired differences from five actor seeds j and three demand multipliers d:

$$\Delta Y_b = \frac{1}{15} \sum_{j=1}^{5} \sum_{d \in \{0.75,1,1.25\}} (Y_{bjd}^{\text{final}} - Y_{bjd}^{\text{ref}}). \tag{17}$$

The same averaging as in (17) gives the completion-rate difference $\Delta R_{\text{cmp},b}$ and unfinished-passenger difference ΔU_b, always proposed minus reference. Unsubscripted differences denote equal-weight means across the ten blocks. Reported overall means give equal weight to the scenario–actor pairs. They are not obtained by pooling passengers across episodes.

We require both a time benefit and completion noninferiority. A time win is $\Delta Y_b < 0$; a completion win is $\Delta R_{\text{cmp},b} > -0.003$, using a 0.3-percentage-point noninferiority margin. For w wins among ten blocks, the one-sided exact sign-test probability is

$$p(w) = 2^{-10} \sum_{\ell=w}^{10} \binom{10}{\ell}. \tag{18}$$

Here ℓ is a possible win count in the binomial tail in (18), distinct from the decision index k. Ties count as nonwins. Each component uses a significance level of 0.025; at least nine wins are required. The intersection–union decision succeeds only when both components pass. The protocol also requires favorable mean and demand-level directions and a complete evaluation matrix.

The sign test assesses directional consistency across independent blocks. It does not provide a confidence interval for the population mean difference. Demand- and actor-specific summaries, waiting/ride decompositions, and comparisons with the other controllers are descriptive.

VI. Results

A. Passenger-Time and Completion Outcomes

Table II reports the primary comparison. Generalized passenger time decreases from 2,484.86 to 2,392.73 s per departed passenger, a mean paired difference of -92.12 s and a descriptive relative reduction of 3.71%. Completion increases by 0.466 percentage points. The mean number of unfinished passengers falls by 77.22 per episode, or 11.78%.

The decomposition makes the passenger tradeoff explicit. Waiting time falls by 56.23 s per departed passenger, while in-vehicle time rises by 20.34 s. With the fixed waiting-time weight,

$$2(-56.23) + 20.34 \approx -92.12 \text{ s}.$$

TABLE II
PRIMARY COMPARISON ON FRESH TEST SCENARIOS

Outcome	Reference	Proposed	Difference
Generalized time (s/pax)	2,484.86	2,392.73	−92.12
Waiting time (s/pax)	720.12	663.88	−56.23
In-vehicle time (s/pax)	1,044.63	1,064.96	+20.34
Completion (%)	96.185	96.650	+0.466 pp
Unfinished (pax/episode)	655.42	578.20	−77.22

Reference: same actor and calibration with a 20-s rule in the later window. Proposed: 60-s rule over the wider window. Time is accrued to the common horizon and divided by all departed passengers. Differences are computed before rounding. The abbreviations pax and pp denote passengers and percentage points, respectively.

TABLE III
PAIRED DIFFERENCES BY DEMAND LEVEL

Demand	ΔY (s/pax)	ΔR_{cmp} (pp)	ΔU (pax)
0.75	−40.29	+0.185	−21.00
1.00	−87.61	+0.470	−70.92
1.25	−148.46	+0.741	−139.74

The proposed controller therefore improves the weighted outcome while imposing a measurable cost on passengers already in vehicles.

Every environment block has lower generalized time and higher completion (Fig. 3). Both prespecified sign tests have ten wins and $p = 2^{-10} = 0.0009766$. All ten completion differences are positive, so meeting the prespecified criterion does not depend on its allowance for small losses.

B. Demand and Training-Seed Variation

The benefit increases with demand (Table III). At demand multiplier 0.75, generalized time falls by 40.29 s per departed passenger; at 1.25, it falls by 148.46 s. Completion improves at every level. The in-vehicle increase remains close to 20 s across the three levels, while the waiting reduction grows from 30.14 to 84.71 s.

The direction also holds for all five trained actors. Their mean time differences range from -87.25 to -97.44 s, and completion differences range from $+0.444$ to $+0.485$ percentage points. These summaries show that the overall direction is not produced by a single training seed.

C. Performance of Other Controllers

Table IV places the matched result alongside all controllers in the evaluation matrix. The proposed controller has the lowest displayed mean generalized time and the highest completion rate. Its mean generalized time is 201.12 s below online-only SAC and 304.24 s below the transit-adapted H2O+ control.

Without hold calibration or the completion rule, the learned policy has a higher completion rate than the matched reference but also higher generalized passenger time. Thus calibration and the earlier rule do not improve every outcome simultaneously. The final rule achieves a better time–completion tradeoff in these scenarios. The other rows change more than one component relative to the proposed controller, so they cannot isolate the contribution of the actor architecture, mixed replay, or stabilization schedule.

D. Action Coverage and Reproducibility

Zero holding accounts for 80.22% of offline actions, supporting its explicit representation in the policy. Long holds are much rarer. Within the narrower $[0.50, 0.75)$ window used by the reference, 128,437 records satisfy the rule conditions, but only 3.69% contain holds of at least 30 s. These counts do not describe the wider window used by the proposed controller. The 60-s rule is therefore an explicit operational choice with limited coverage in the offline data.

The released records identify the controller, actor seed, checkpoint, scenario, runtime, and result for each episode. An independent calculation reproduces the primary means, demand and block summaries, and sign-test outcomes. The source code and compact experiment records are available in the project repository.[1] Large replay files, checkpoints, and generated scenario inputs are not included in the repository. Consequently, the public records support checking the reported calculations, while rerunning training also requires the original inputs.

VII. DISCUSSION

A. Why Extra Holding Can Improve Passenger Service

Holding is usually motivated by restoring headways. Here it also preserves a boarding opportunity. A lightly loaded bus without an active follower can remain at the stop while additional eligible passengers arrive. The rule permits this only during a specified window and at loads no greater than one quarter of capacity. Its benefit is operationally plausible because boarding remains possible during the additional hold.

The waiting/ride decomposition supports that interpretation: all ten blocks show less waiting and more in-vehicle time, with a favorable weighted total. The larger waiting reduction at higher demand is also consistent with more passengers benefiting from the retained opportunity. However, the aggregate comparison does not identify which particular boarding or transfer events caused the completion gain. It also changes the activation window and hold duration together.

The learned policy still determines proposals outside the rule and contributes through the shared calibration. The completion rule modifies its action only under the specified conditions.

B. What the Evidence Supports

The independent test scenarios show that extending the completion rule reduces generalized passenger time and meets the prespecified completion criterion for the five fixed policies under the tested operating conditions. The sign test supports consistency across environment blocks; it does not quantify uncertainty in the population-average effect. With only ten blocks, the distribution of effect sizes remains uncertain.

The experiment does not establish which learning components are responsible for the controller's performance or whether the learned policy is necessary. The additional comparisons change multiple components, and online-only SAC receives fewer total learner updates because it lacks offline

[1] https://github.com/erzhu419/sumo-rl/tree/berth; the bus-control study is in its `H2Oplus` submodule.

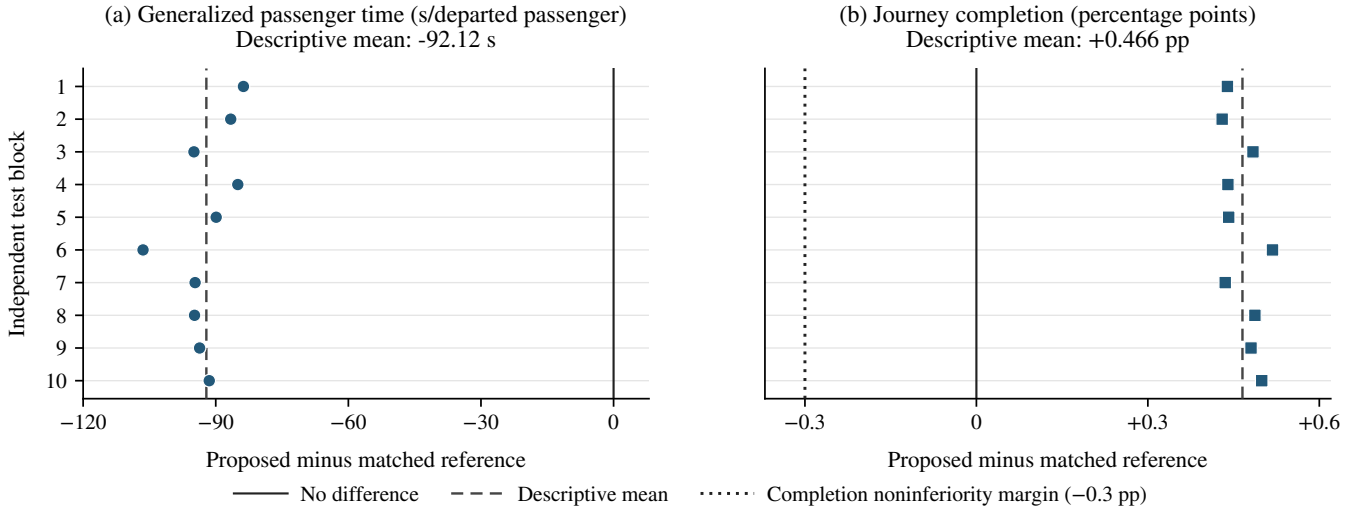


Fig. 3. Paired effects in the ten independent test blocks. Each point averages three demand levels and five actor seeds, with proposed minus reference as the difference. Negative generalized time and positive completion are favorable. The completion noninferiority boundary is -0.3 percentage points. Mean lines are descriptive; no confidence interval for the mean is implied.

TABLE IV
DESCRIPTIVE OUTCOMES FOR ALL NINE EVALUATED CONTROLLERS

Controller	Distinguishing feature	Y (s/pax)	Completion (%)
Completion-aware controller	Final trained policy, hold calibration, and extended completion rule	**2,392.73**	**96.650**
Matched reference	Same actor and calibration, 20-s later-window rule	2,484.86	96.185
Learned policy alone	Final policy without hold calibration or the completion rule	2,623.19	96.453
Transit-adapted H2O+	Hybrid learner with active dynamics weights	2,696.98	96.288
Online-only SAC	Simulator learning without offline initialization	2,593.85	96.232
Actor updates from simulator data	Actor uses simulator replay after the initial phase	2,532.20	95.894
Delayed actor updates	Actor and temperature stay fixed through 10,000 events	2,569.68	96.023
Gradual increase in actor learning rate	Adds an actor learning-rate ramp	2,585.56	96.194
Actor updates with parameter anchoring	Adds interpolation toward the earlier actor	2,581.26	96.053

Each row averages 150 evaluation episodes. The last four rows are development variants evaluated on the same test scenarios. The prespecified joint test compares the proposed controller with the matched reference. Other rows differ in learning or execution and are descriptive. SAC matches the nominal online-event budget, not the hybrid learner's total update count.

initialization. Conventional headway control with and without the same completion rule is an important missing comparison. These limitations prevent a general claim of superiority over existing holding methods.

C. Transfer and Operational Limits

Both the source records and evaluation outcomes are simulated. The event-based training model passes the specified agreement checks, but this does not establish transfer to observed operations, another network, or every policy-induced state distribution. Logged and evaluation SUMO versions also differ. A paired evaluation of the same fixed controllers under both versions would help separate software-version effects from other dynamics differences.

The observation compresses histories and future service availability. In particular, an inactive following bus is not necessarily the last scheduled service. The arrival-rate input is constructed from demand and frequency information, and the rule does not predict terminal shortfall explicitly. Its fixed load threshold and time window may need adjustment for other timetables.

Completion is assessed at a finite horizon, and the reward includes only accrued time. A future controller could model terminal shortfall directly or learn a constrained policy, provided its data cover the relevant states and long holding actions. The current physical cap provides a bounded action, not a general operational safety guarantee. Field evaluation would also need dispatch, driver, accessibility, and connection requirements that are outside this benchmark.

VIII. CONCLUSION

This paper develops a bus holding controller that learns from microscopic simulation records and further interaction in a faster event-based simulator. A shared policy represents exact zero hold, accounts for passenger time across asynchronous events, and supplies a holding proposal. A deterministic completion rule then preserves selected boarding opportunities for lightly loaded buses without an active follower.

On independent test scenarios, replacing an earlier narrow-window rule with the proposed rule reduces generalized passenger time by 92.12 s per departed passenger and increases completion by 0.466 percentage points. The benefit consists of a 56.23-s waiting reduction and a 20.34-s in-vehicle increase. Both prespecified criteria hold in all ten independent blocks. The study supports this rule change for the tested hybrid-learned controller. Comparisons with conventional holding policies under the same completion rule, followed by operational validation, are needed to assess wider applicability.

ACKNOWLEDGMENT

OpenAI ChatGPT assisted with restructuring and rewriting the abstract, Sections I–VIII, and figure captions from the existing manuscript, source code, and experiment records, and with preparing the schematic and plots from those records.

REFERENCES

[1] M. Rezazada, N. Nassir, E. Tanin, and A. Ceder, "Bus bunching: A comprehensive review from demand, supply, and decision-making perspectives," *Transport Reviews*, vol. 44, no. 4, pp. 766–790, 2024.

[2] C. F. Daganzo, "A headway-based approach to eliminate bus bunching: Systematic analysis and comparisons," *Transportation Research Part B: Methodological*, vol. 43, no. 10, pp. 913–921, 2009.

[3] F. Delgado, J. C. Muñoz, and R. Giesen, "How much can holding and/or limiting boarding improve transit performance?" *Transportation Research Part B: Methodological*, vol. 46, no. 9, pp. 1202–1217, 2012.

[4] F. Alesiani and K. Gkiotsalitis, "Reinforcement learning-based bus holding for high-frequency services," in *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*. IEEE, 2018, pp. 3162–3168.

[5] J. Wang and L. Sun, "Dynamic holding control to avoid bus bunching: A multi-agent deep reinforcement learning framework," *Transportation Research Part C: Emerging Technologies*, vol. 116, p. 102661, 2020.

[6] ——, "Reducing bus bunching with asynchronous multi-agent reinforcement learning," *arXiv preprint arXiv:2105.00376*, 2021. [Online]. Available: https://arxiv.org/abs/2105.00376

[7] ——, "Multi-objective multi-agent deep reinforcement learning to reduce bus bunching for multiline services with a shared corridor," *Transportation Research Part C: Emerging Technologies*, vol. 155, p. 104309, 2023.

[8] H. Niu, S. Sharma, Y. Qiu, M. Li, G. Zhou, J. Hu, and X. Zhan, "When to trust your simulator: Dynamics-aware hybrid offline-and-online reinforcement learning," *arXiv preprint arXiv:2206.13464*, 2022. [Online]. Available: https://arxiv.org/abs/2206.13464

[9] H. Niu, T. Ji, B. Liu, H. Zhao, X. Zhu, J. Zheng, P. Huang, G. Zhou, J. Hu, and X. Zhan, "H2O+: An improved framework for hybrid offline-and-online RL with dynamics gaps," *arXiv preprint arXiv:2309.12716*, 2023. [Online]. Available: https://arxiv.org/abs/2309.12716

[10] Y. Xuan, J. Argote, and C. F. Daganzo, "Dynamic bus holding strategies for schedule reliability: Optimal linear control and performance analysis," *Transportation Research Part B: Methodological*, vol. 45, no. 10, pp. 1831–1845, 2011.

[11] G. E. Sánchez-Martínez, H. N. Koutsopoulos, and N. H. M. Wilson, "Real-time holding control for high-frequency transit with dynamics," *Transportation Research Part B: Methodological*, vol. 83, pp. 1–19, 2016.

[12] M. Andres and R. Nair, "A predictive-control framework to address bus bunching," *Transportation Research Part B: Methodological*, vol. 104, pp. 123–148, 2017.

[13] J. Wang and L. Sun, "Robust dynamic bus control: A distributional multi-agent reinforcement learning approach," *arXiv preprint arXiv:2111.01946*, 2021. [Online]. Available: https://arxiv.org/abs/2111.01946

[14] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," *arXiv preprint arXiv:1801.01290*, 2018. [Online]. Available: https://arxiv.org/abs/1801.01290

[15] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, and S. Levine, "Soft actor-critic algorithms and applications," *arXiv preprint arXiv:1812.05905*, 2018. [Online]. Available: https://arxiv.org/abs/1812.05905

[16] A. Kumar, A. Zhou, G. Tucker, and S. Levine, "Conservative Q-learning for offline reinforcement learning," *arXiv preprint arXiv:2006.04779*, 2020. [Online]. Available: https://arxiv.org/abs/2006.04779

[17] I. Kostrikov, A. Nair, and S. Levine, "Offline reinforcement learning with implicit Q-learning," *arXiv preprint arXiv:2110.06169*, 2021. [Online]. Available: https://arxiv.org/abs/2110.06169

[18] S. Lee, Y. Seo, K. Lee, P. Abbeel, and J. Shin, "Offline-to-online reinforcement learning via balanced replay and pessimistic Q-ensemble," *arXiv preprint arXiv:2107.00591*, 2021. [Online]. Available: https://arxiv.org/abs/2107.00591

[19] M. Nakamoto, Y. Zhai, A. Singh, M. S. Mark, Y. Ma, C. Finn, A. Kumar, and S. Levine, "Cal-QL: Calibrated offline RL pre-training for efficient online fine-tuning," *arXiv preprint arXiv:2303.05479*, 2023. [Online]. Available: https://arxiv.org/abs/2303.05479

[20] H. Zhang, W. Xu, and H. Yu, "Policy expansion for bridging offline-to-online reinforcement learning," *arXiv preprint arXiv:2302.00935*, 2023. [Online]. Available: https://arxiv.org/abs/2302.00935

[21] P. J. Ball, L. Smith, I. Kostrikov, and S. Levine, "Efficient online reinforcement learning with offline data," *arXiv preprint arXiv:2302.02948*, 2023. [Online]. Available: https://arxiv.org/abs/2302.02948

[22] Z. Zhou, A. Peng, Q. Li, S. Levine, and A. Kumar, "Efficient online reinforcement learning fine-tuning need not retain offline data," *arXiv preprint arXiv:2412.07762*, 2024. [Online]. Available: https://arxiv.org/abs/2412.07762

[23] Y. Hu, T. Dong, and S. Li, "Coordinating ride-pooling with public transit using reward-guided conservative Q-learning: An offline training and online fine-tuning reinforcement learning framework," *arXiv preprint arXiv:2501.14199*, 2025. [Online]. Available: https://arxiv.org/abs/2501.14199

[24] B. Eysenbach, S. Asawa, S. Chaudhari, S. Levine, and R. Salakhutdinov, "Off-dynamics reinforcement learning: Training for transfer with domain classifiers," *arXiv preprint arXiv:2006.13916*, 2020. [Online]. Available: https://arxiv.org/abs/2006.13916

[25] R. S. Sutton, D. Precup, and S. Singh, "Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning," *Artificial Intelligence*, vol. 112, no. 1–2, pp. 181–211, 1999.

[26] P. Alvarez Lopez, M. Behrisch, L. Bieker-Walz, J. Erdmann, Y.-P. Flötteröd, R. Hilbrich, L. Lücken, J. Rummel, P. Wagner, and E. Wießner, "Microscopic traffic simulation using SUMO," in *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*. IEEE, 2018, pp. 2575–2582.